

418 - I'm a Teafault

For over a decade, Linux has allowed you to handle page faults in userspace¹. So, following the old "because why not" rule, we're going to resolve faults with a little help from a low-level tool: the HTTP protocol. Maybe this could be useful in some simple cases involving sequential access, where you can prefetch to amortize latency. While supporting offsets within a page would be easy, tree-like structures or other pointer-chasing-ish stuff would be much harder.

The server

```
from http.server import BaseHTTPRequestHandler,
    ThreadingHTTPServer
class Handler(BaseHTTPRequestHandler):
    def do_GET(self):
        page_number = int(self.path[len("/page/"):])
        data = f"P-page:{page_number}\0".encode()
        remaining = 4096 - len(data)
        data += (b"AB" * ((remaining + 1) // 2))[:remaining]
        self.send_response(200)
        self.send_header("Content-Length", str(len(data)))
        self.end_headers(); self.wfile.write(data)
ThreadingHTTPServer(("", 8080), Handler).
    serve_forever()
```

The client

```
#include <arpa/inet.h>
#include <fcntl.h>
#include <linux/userfaultfd.h>
#include <poll.h>
#include <pthread.h>
#include <stdio.h>
#include <string.h>
#include <sys/ioctl.h>
#include <sys/mman.h>
#include <sys/syscall.h>
#include <unistd.h>
#define PAGE_SIZE 4096
#define NUM_PAGES 256
static int uffd; static char *region;
int http_get(int page, char *buffer){
    int socket_fd = socket(AF_INET, SOCK_STREAM, 0), n;
    struct sockaddr_in addr = {AF_INET, htons(8080)};
    inet_pton(AF_INET, "127.0.0.1", &addr.sin_addr);
    connect(socket_fd, (void *)&addr, sizeof addr);
    char request[128], response[8192];
    snprintf(request, sizeof request, "GET /page/%d HTTP/1.0\r\n" "Host: " "127.0.0.1" "\r\n\r\n", page);
    write(socket_fd, request, strlen(request));
    size_t total = 0;
    while ((n = read(socket_fd, response + total, sizeof response - total - 1)) > 0) total += n;
    close(socket_fd);
    response[total] = 0;
    char *body = strstr(response, "\r\n\r\n") + 4;
    memcpy(buffer, body, total - (body - response));
    return 0;
}
void *fault_handler(void *arg){
```

```
    struct pollfd pollfd;
    char page[PAGE_SIZE];
    for (;;) {
        pollfd = (struct pollfd){uffd, POLLIN, 0};
        poll(&pollfd, 1, -1);
        struct uffd_msg msg;
        read(uffd, &msg, sizeof msg);
        unsigned long fault_address = msg.arg.pagefault.address;
        unsigned long page_address = fault_address & ~(PAGE_SIZE - 1);
        int page_number = (page_address - (unsigned long)region) / PAGE_SIZE;
        printf("PAGE FAULT: page=%d\n", page_number);
        http_get(page_number, page);
        struct uffdio_copy copy = {
            .src = (unsigned long)page,
            .dst = page_address,
            .len = PAGE_SIZE
        };
        ioctl(uffd, UFFDIO_COPY, &copy);
    }
}
int main(void){
    size_t length = NUM_PAGES * PAGE_SIZE;
    uffd = syscall(SYS_userfaultfd, 0_CLOEXEC | 0_NONBLOCK | UFFD_USER_MODE_ONLY);
    struct uffdio_api api = {.api = UFFD_API};
    ioctl(uffd, UFFDIO_API, &api);
    region = mmap(NULL, length, PROT_READ | PROT_WRITE, MAP_PRIVATE | MAP_ANONYMOUS, -1, 0);
    struct uffdio_register reg = {
        .range = {(unsigned long)region, length},
        .mode = UFFDIO_REGISTER_MODE_MISSING
    };
    ioctl(uffd, UFFDIO_REGISTER, &reg);
    pthread_t thread;
    pthread_create(&thread, NULL, fault_handler, NULL);
    pthread_detach(thread);
    for (int page = 0; page < NUM_PAGES; page++) {
        printf("Reading page %d...\n", page);
        char *page_start = region + page * PAGE_SIZE;
        volatile char value = *page_start;
        // *(region + page * PAGE_SIZE) = 'X';
        printf("value = %s\n", page_start);
        madvise(page_start, PAGE_SIZE, MADV_DONTNEED);
    }
    volatile unsigned char value = region[0];
    printf("Reading page 0 again...\n");
    printf("value = %s\n", region);
    return 0;
}
```

Funny trap, though. Using `printf()` in the fault handler means that if another `printf()` triggers a page fault, it can deadlock on the `stdout` lock. That's why we touch the page before printing.

Under pressure

Try running it with a tiny memory limit:

```
python3 ./server.py
gcc -pthread client.c -o client && systemd-run
--user --scope -p MemoryMax=1M -p
MemorySwapMax=0 ./client
```

The HTTP server has somehow become a rather questionable backing store :) The code, with comments and minimal error handling, is available at <https://github.com/artur-augus-tyniak/418-teafault>.

¹ man 2 userfaultfd